

LA Lab - 8

Name : Vedant Shimpi

Reg ID : 241080068

Branch : IT

Aim : Network File Systems (NFS): Set up an NFS server and client, exploring different mount options and performance optimization.

Theory :

Network File System (NFS) is a distributed file system protocol that allows a user on a client machine to access files over a network in a way similar to how local storage is accessed. It was originally developed by Sun Microsystems and is now widely used in Linux/Unix environments.

NFS works on a client-server architecture:

- The NFS Server shares directories (called *exports*)
- The NFS Client mounts these directories and accesses them as part of its local file system

Working Principle

1. The server defines shared directories in a configuration file (e.g., `/etc/exports`)
2. The client requests access to these directories
3. The server grants permission based on rules (IP, hostname, etc.)
4. The client mounts the shared directory using the mount command
5. Files are accessed transparently as if they are local

Key Features of NFS

- Remote access: Access files across a network
 - Transparency: Remote files appear local
 - Centralized storage: Data stored on one server
 - Multi-user access: Multiple clients can access simultaneously
 - Security support: Authentication and permission control
-

Common NFS Mount Options

Mount options control performance and behavior:

Performance-related

- rsize / wsize → Define read/write buffer size
- async → Faster writes (less safe)
- sync → Safer but slower

Reliability-related

- hard → Keeps retrying if server fails
- soft → Stops retrying after timeout

Timeout & Retry

- timeo → Timeout duration
- retrans → Number of retries

Access Control

- ro → Read-only
 - rw → Read-write
-

Performance Optimization Techniques

- Increase rsize and wsize for faster data transfer
 - Use async for high-speed environments (with caution)
 - Use dedicated network (LAN) to reduce latency
 - Enable caching mechanisms
 - Use newer NFS versions (like NFSv4) for better performance and security
 - Optimize server hardware (SSD, RAM)
-

Advantages of NFS

- Easy file sharing across systems
 - Reduces storage duplication
 - Centralized management
 - Scalable for large networks
-

Limitations of NFS

- Performance depends on network speed
 - Security risks if not configured properly
 - Server failure affects all clients
 - Less efficient over WAN compared to LAN
-

Output :

Task 1 — Install & Configure NFS Server

Install NFS and export a shared directory.

1. Install tools: `sudo pacman -S nfs-utils`

2. Create the share: `sudo mkdir -p /srv/nfs/share` `sudo chown nobody:nobody /srv/nfs/share` (NFS often uses 'nobody' for security) `sudo chmod 777 /srv/nfs/share`
3. Configure Exports: Open the exports file: `sudo nano /etc/exports` Add this line: `/srv/nfs/share 127.0.0.1(rw,sync,no_subtree_check)`
4. Start the Server: `sudo systemctl enable --now nfs-server`

```
└─ sudo pacman -S nfs-utils && sudo mkdir -p /srv/nfs/share
[sudo] password for archuserbtw:
resolving dependencies...
looking for conflicting packages...

Packages (4) ding-lib-0.7.0-1  gssproxy-0.9.2-3  nfsidmap-2.9.1-1  nfs-utils-2.9.1-1

Total Download Size:   0.61 MiB
Total Installed Size:  2.01 MiB

:: Proceed with installation? [Y/n] y
:: Retrieving packages...
nfsidmap-2.9.1-1-x86_64      46.4 KiB  85.4 KiB/s  00:01 [#####] 100%
ding-lib-0.7.0-1-x86_64    73.8 KiB  132 KiB/s  00:01 [#####] 100%
gssproxy-0.9.2-3-x86_64    96.8 KiB  173 KiB/s  00:01 [#####] 100%
nfs-utils-2.9.1-1-x86_64  404.7 KiB 703 KiB/s  00:01 [#####] 100%
Total (4/4)                621.7 KiB 1048 KiB/s  00:01 [#####] 100%
(4/4) checking keys in keyring [#####] 100%
(4/4) checking package integrity [#####] 100%
(4/4) loading package files [#####] 100%
(4/4) checking for file conflicts [#####] 100%
(4/4) checking available disk space [#####] 100%
:: Processing package changes...
(1/4) installing nfsidmap [#####] 100%
(2/4) installing ding-lib [#####] 100%
(3/4) installing gssproxy [#####] 100%
(4/4) installing nfs-utils [#####] 100%
Optional dependencies for nfs-utils
  sqlite: for nfsdcltrack and fsidd usage [installed]
  python: for rpcctl, nfsiostat, nfsdclns and mountstats usage [installed]
:: Running post-transaction hooks...
(1/5) Creating system user accounts...
Creating group 'rpcuser' with GID 34.
Creating user 'rpcuser' (RPC Service User) with UID 34 and GID 34.
(2/5) Reloading system manager configuration...
(3/5) Reloading user manager configuration...
(4/5) Reloading device manager configuration...
(5/5) Arming ConditionNeedsUpdate...

┌─ A ~ ..... ✓ took 7s at 21:01:51
└─ sudo chown nobody:nobody /srv/nfs/share && sudo chmod 777 /srv/nfs/share

┌─ A ~ ..... ✓ at 21:02:32
└─ sudo nano /etc/exports
```

```
GNU nano 9.0 /etc/exports Modified
# /etc/exports - exports(5) - directories exported to NFS clients
#
# Example for NFSv3:
# /srv/home      hostname1(rw,sync) hostname2(ro,sync)
# Example for NFSv4:
# /srv/nfs4      hostname1(rw,sync,fsid=0)
# /srv/nfs4/home hostname1(rw,sync,nohide)
# Using Kerberos and integrity checking:
# /srv/nfs4      *(rw,sync,sec=krb5i,fsid=0)
# /srv/nfs4/home *(rw,sync,sec=krb5i,nohide)
#
# Use `exportfs -arv` to reload.
/srv/nfs/share 127.0.0.1(rw,sync,no_subtree_check)
```

```
sudo systemctl enable --now nfs-server
[sudo] password for archuserbtw:
Created symlink '/etc/systemd/system/multi-user.target.wants/nfs-server.service' → '/usr/lib/systemd/system/nfs-server.service'
```

Task 2 — Mount NFS Share on Client

Install NFS client tools and mount the remote share.

Now, let's act as the client and "reach out" to the server we just built.

1. Create a mount point: **sudo mkdir -p /mnt/nfs_client**
2. Mount it: **sudo mount -t nfs 127.0.0.1:/srv/nfs/share /mnt/nfs_client**
3. Verify: Run **df -h** and look for the NFS entry.

```
sudo mkdir -p /mnt/nfs_client

sudo mount -t nfs 127.0.0.1:/srv/nfs/share /mnt/nfs_client

df -h | grep "nfs"
127.0.0.1:/srv/nfs/share 198G 112G 76G 60% /mnt/nfs_client
```

Task 3 — Persistent Mount via /etc/fstab

Make the NFS mount survive reboots.

To make it survive a reboot, add this to your /etc/fstab: 127.0.0.1:/srv/nfs/share /mnt/nfs_client nfs defaults,_netdev 0 0
Note: _netdev is critical; it tells Linux to wait for the network to be up before trying to mount this.

```
GNU nano 9.0 /etc/fstab
# Static information about the filesystems.
# See fstab(5) for details.

# <file system> <dir> <type> <options> <dump> <pass>
# /dev/nvme0n1p5
UUID=81e23db4-3fc3-4397-b886-1469380d6b8d / ext4 rw,relatime
# /dev/nvme0n1p7
UUID=E2FA-6A7D /boot vfat rw,relatime,fmask=0022,dmask=0022,codepage=
# /dev/nvme0n1p6
UUID=296c1789-fcfb-4d40-bd0c-563df456b9b1 /mnt/test_bench ext4 defaults 0 2
# nfs server
127.0.0.1:/srv/nfs/share /mnt/nfs_client nfs defaults,_netdev 0 0
```

Task 4 — Explore Export Security Options

Test different security configurations on the server.

Try changing the options in /etc/exports:

- **Read Only:** Change rw to ro. Run `sudo exportfs -arv` to apply. Try to create a file in the client folder; it should fail.
- **Root Squash:** The default `root_squash` prevents a client's root user from having root power on the server. Test this by creating a file as `sudo` and checking the owner on the server side (`ls -l /srv/nfs/share`).

```
GNU nano 9.0 /etc/exports
# /etc/exports - exports(5) - directories exported to NFS clients
#
# Example for NFSv3:
# /srv/home      hostname1(rw,sync) hostname2(ro,sync)
# Example for NFSv4:
# /srv/nfs4      hostname1(rw,sync,fsid=0)
# /srv/nfs4/home hostname1(rw,sync,nohide)
# Using Kerberos and integrity checking:
# /srv/nfs4      *(rw,sync,sec=krb5i,fsid=0)
# /srv/nfs4/home *(rw,sync,sec=krb5i,nohide)
#
# Use `exportfs -arv` to reload.
/srv/nfs/share 127.0.0.1(ro,sync,no_subtree_check)
```

```
sudo nano /etc/fstab
sudo nano /etc/exports
sudo exportfs -arv
exporting 127.0.0.1:/srv/nfs/share
sudo touch /mnt/nfs_client/a.txt
touch: cannot touch '/mnt/nfs_client/a.txt': Read-only file system
```

Task 5 — Performance Testing & Optimization

Benchmark NFS and tune for better throughput

1. Check stats: `nfsstat -c` (Client stats) or `nfsstat -s` (Server stats).
2. Optimization: Try mounting with specific buffer sizes: `sudo mount -t nfs -o rsize=32768,wsize=32768 127.0.0.1:/srv/nfs/share /mnt/nfs_client`.

```
~  
- sudo mount -t nfs -o rsize=16384,wsiz=16384 127.0.0.1:/srv/nfs/share /mnt/nfs_client
```

```
~  
- sudo systemctl daemon-reload
```

```
~  
- dd if=/dev/zero of=/mnt/nfs_client/test_file bs=1M count=500 conv=fsync  
500+0 records in  
500+0 records out  
524288000 bytes (524 MB, 500 MiB) copied, 31.8461 s, 16.5 MB/s
```

```
~  
- cat /mnt/nfs_client/test_file > /dev/null
```

```
~  
- dd if=/dev/zero of=/mnt/nfs_client/test_file bs=1M count=500 conv=fsync  
500+0 records in  
500+0 records out  
524288000 bytes (524 MB, 500 MiB) copied, 1.03354 s, 507 MB/s
```

```
~  
- nfsstat -c  
Client rpc stats:  
calls      retrans    authrefrsh  
32126      0          32138  
  
Client nfs v4:  
null      read      write     commit    open  
1          0%        0          0%        31999    99%      2          0%      2          0%  
open_conf  open_noat open_dgrd  close     setattr  
0          0%        0          0%        0         0%      0          0%      0          0%  
fsinfo     renew     setclntid confirm    lock  
12         0%        0          0%        0         0%      0          0%      0          0%  
lockt      locku     access    getattr   lookup  
0          0%        0          0%        13        0%      19         0%      24         0%  
lookup_root remove    rename     link      symlink  
3          0%        0          0%        0         0%      0          0%      0          0%  
create     pathconf statfs     readlink  readdir  
0          0%        12         0%        4         0%      0          0%      0          0%  
server_caps delegreturn getacl     setacl    fs_locations  
24         0%        2          0%        0         0%      0          0%      0          0%  
rel_lkowner secinfo    fsid_present exchange_id create_session  
0          0%        0          0%        0         0%      2          0%      1          0%  
destroy_session sequence  get_lease_time reclaim_comp layoutget  
0          0%        11         0%        0         0%      1          0%      0          0%  
getdevinfo layoutcommit layoutreturn secinfo_no test_stateid  
0          0%        0          0%        0         0%      3          0%      0          0%  
free_stateid getdevicelist bind_conn_to_ses destroy_clientid seek  
0          0%        0          0%        0         0%      0          0%      0          0%  
allocate   deallocate layoutstats clone  
0          0%        0          0%        0         0%      0          0%
```

```
cat /mnt/nfs_client/test_file > /dev/null

nfsstat -c
Client rpc stats:
calls      retrans    authrefrsh
64122      0           64138

Client nfs v4:
null      read      write     commit    open
1 0% 0 0% 63999 99% 3 0% 2 0%
open_conf open_noat open_dgrd close      setattr
0 0% 0 0% 0 0% 0 0% 1 0%
fsinfo    renew    setclntid confirm    lock
12 0% 0 0% 0 0% 0 0% 0 0%
lockt     locku    access    getattr    lookup
0 0% 0 0% 13 0% 20 0% 24 0%
lookup_root remove    rename    link        symlink
3 0% 0 0% 0 0% 0 0% 0 0%
create    pathconf statfs     readlink    readdir
0 0% 12 0% 4 0% 0 0% 0 0%
server_caps delegreturn getacl     setacl       fs_locations
24 0% 2 0% 0 0% 0 0% 0 0%
rel_lkowner secinfo    fsid_present exchange_id create_session
0 0% 0 0% 0 0% 2 0% 1 0%
destroy_session sequence    get_lease_time reclaim_comp layoutget
0 0% 11 0% 0 0% 1 0% 0 0%
getdevinfo layoutcommit layoutreturn secinfo_no test_stateid
0 0% 0 0% 0 0% 3 0% 0 0%
free_stateid getdevicelist bind_conn_to_ses destroy_clientid seek
0 0% 0 0% 0 0% 0 0% 0 0%
allocate    deallocate layoutstats clone
0 0% 0 0% 0 0% 0 0%
```

```
sudo mount -t nfs -o rsize=1024,wsize=1024 127.0.0.1:/srv/nfs/share /mnt/nfs_client

sudo systemctl daemon-reload
```

```
dd if=/dev/zero of=/mnt/nfs_client/test_file bs=1M count=500 conv=fsync
500+0 records in
500+0 records out
524288000 bytes (524 MB, 500 MiB) copied, 16.9414 s, 30.9 MB/s

nfsstat -c
Client rpc stats:
calls      retrans    authrefrsh
576095      0          576162

Client nfs v4:
null      read      write     commit    open
1          0%        0         0%        575977    99%      4          0%      3
open_conf open_noat open_dgrd close      0%        1          0%      1
0          0%        0         0%        0          0%        0          0%      0
fsinfo    renew     setclntid confirm    0          0%        0          0%      0
lockt     locku     access    getattr   24         0%        30         0%      0
0          0%        0         0%        17         0%        0          0%      0
lookup_root remove    rename     link       0          0%        0          0%      0
4          0%        0         0%        0          0%        0          0%      0
create    pathconf statfs     readlink   0          0%        0          0%      0
0          0%        16        0%        4          0%        0          0%      0
server_caps delegreturn getacl     setacl     0          0%        0          0%      0
32         0%        3          0%        0          0%        0          0%      0
rel_lkowner secinfo    fsid_present exchange_id create_session
0          0%        0          0%        0          0%        2          0%      1
destroy_session sequence   get_lease_time reclaim_comp layoutget
0          0%        12         0%        0          0%        1          0%      0
0          0%        0          0%        0          0%        0          0%      0
getdevinfo layoutcommit layoutreturn secinfo_no test_stateid
0          0%        0          0%        0          0%        4          0%      0
free_stateid getdevicelist bind_conn_to_ses destroy_clientid seek
0          0%        0          0%        0          0%        0          0%      0
0          0%        0          0%        0          0%        0          0%      0
allocate  deallocate layoutstats clone
0          0%        0          0%        0          0%        0          0%      0
```

Conclusion : In this experiment, we successfully set up an NFS server and client to enable file sharing over a network. The study demonstrated how remote directories can be mounted and accessed as local storage, making file management efficient and centralized.

Different mount options were explored to understand their impact on performance, reliability, and security. It was observed that parameters such as

buffer size (rsize, wsize) and mount modes (sync, async, hard, soft) significantly influence system behavior.

Overall, NFS proves to be a powerful and flexible solution for distributed file sharing, especially in LAN environments. However, proper configuration and optimization are essential to ensure high performance, reliability, and security.
